NLP Pipeline Report: Cairo Restaurant Reviews

Author: Yassin | **Project:** Milestone 1, Data Preprocessing Pipeline | **Data:** Google Maps reviews for Fifth Settlement, Rehab and First Settlement (Cairo), scraped 2026-10-25

1. What I built

My part of the milestone is the NLP preprocessing pipeline. It takes the two raw review files, keeps only reviews with written text, cleans the text in layers, labels the language, creates weak sentiment labels from stars, and prepares a model-ready dataset with a place-based split. I also built a second set of optional experiments (Pipeline v2).

Design rule: the original text (`text_raw`) is never modified. Every lossy step writes to a new column, so no useful information is thrown away automatically, as the milestone description requires.

Item	Value
Reviews loaded (all, with or without text)	6,120
Reviews with written text	2,855 (47%)
Usable reviews (3+ words, not duplicate, language not none/other)	2,415
Language mix	Arabic 1,553, English/Latin 1,252, mixed 16, Arabizi 9, none (emoji or digits only) 20, other 5
Weak sentiment labels (from stars)	positive 2,037, negative 706, neutral 112
Split	train / validation / test (about 70/15/15) by place, with 0 places shared between train and test

The 3,265 reviews without text are not wasted: they still have star ratings, so the notebook keeps all 6,120 for ratings and scores (`df_all` , `place_scores`) and uses only the 2,855 for text processing.

2. Techniques used in the pipeline (v1)

#	Operation	What it does	Why it fits this dataset	Lossy?
1	Noise and owner-reply removal	Removes URLs, phone-like numbers, invisible characters and odd symbols, collapses whitespace and newlines, cuts off an appended machine translation, and checks whether a review is only a copy of the owner reply	Google reviews contain noise. Owner replies are stored in a separate field, so I verified none leaked into the text (0 found). Emojis are kept.	Slightly
2	Arabic normalization	Unifies alef variants (أ إ آ to ا), turns ى into ي, removes diacritics and tatweel. The ة to ه change is available but off by default.	Egyptian Arabic spelling is inconsistent, so one word appears in many forms	Yes (small)
3	Repeated-letter reduction	Arabic: 3 or more of the same letter become 1. English: 3 or more become 2. A flag `has_elongation` keeps the emphasis signal.	Elongation is emphasis and splits the vocabulary. The flag keeps the sentiment information.	Yes (flagged)
4	Emoji handling	Keeps emojis in `text_clean`, and writes a second version with emojis converted to words	Emojis carry strong sentiment, and many tokenizers drop them or map them to unknown tokens	No (separate column)

#	Operation	What it does	Why it fits this dataset	Lossy?
5	Language detection	Labels each review as Arabic, English, mixed, Arabizi, none or other using script ratios, plus a heuristic for Arabizi (digits such as 3, 5, 7 inside Latin words)	The corpus is bilingual, so later models may need per-language handling	No

Deliberately not applied: stopword removal, stemming and emoji stripping on the main text, because they remove useful sentiment information (for example, "not good").

Additional steps for the modeling stage

Step	Output column	Purpose
Weak sentiment labels (1-2 stars negative, 3 neutral, 4-5 positive)	`sentiment`	Gives Milestone 2 labels and a class distribution
Model-ready text (lowercase, emojis as words)	`text_model`	One consistent version for models
Tokenization (Arabic and Latin words)	`tokens` , `n_tokens`	Token distribution and TF-IDF
Optional stopword reduction that keeps negations (not, no, مش, غير)	`text_tfidf`	For TF-IDF only. It is separate because it is lossy.
Usability flag	`usable`	Flags short, duplicate and non-Arabic/English reviews without deleting them
Place-based split	`split`	A restaurant's reviews never appear in both train and test
Final clean text (text_model without punctuation)	`text_final`	The single column I recommend as the clean text

3. Five before / after examples (real reviews)

Example 1: appended machine translation removed (noise removal)

- Before (811 characters): the Arabic review, a blank line, then an English translation that starts "The food is somewhat okay and acceptable, but unfortunately the service is extremely poor..."
- After: only the Arabic text remains, ending in "...فلا يستحق الزيارة بتاتا"
- Why: the scraper stored an English machine translation inside the `text` field of some Arabic reviews. If it stays, language detection and the vocabulary are wrong (those reviews look mixed Arabic/English). 7 reviews were affected.

Example 2: repeated letters (Arabic)

- Before: `احلى مطعم ف العالم الخدمه ممتازه والاكل خطيررررر`
- After: `احلي مطعم ف العالم الخدمه ممتازه والاكل خطير`
- Why: `خطيررررر` becomes `خطير`, so the elongated form joins the normal word. `احلى` becomes `احلي` (ى to ي). `has_elongation` is set to true so the emphasis is not lost.

Example 3: Arabic normalization and punctuation

- Before: `الاكل أسوأ ما يكون ، الأحجام صغيرة. الاكل اتأخر جداً.`
- After (`text_clean`): `الاكل اسوا ما يكون ، الاحجام صغيرة. الاكل اتاخر جدا.`
- After (`text_final`): `الاكل اسوا ما يكون الاحجام صغيرة الاكل اتاخر جدا`
- Why: alef variants (أ) and the diacritic on `جداً` are unified, so spelling variants map to one form. `text_final` also removes punctuation.

Example 4: noise, English elongation and emoji

- Before: `Hossssaaaaaam is the goat cashier in chery shop. 1515151515151515 🐐🐐🐐🐐🐐`
- After (`text_clean`): `Hossaam is the goat cashier in chery shop. 🐐🐐🐐🐐🐐`
- After (`text_final`): `hossaam is the goat cashier in chery shop goat goat goat goat goat`
- Why: the long digit string (phone-like noise) is removed, the English letter repeats are capped at 2, and the emojis are converted to words so a text-only model can use them.

Example 5: Arabizi detection

- Before: `2a da bgad aw7ash 7aga do2taha f 7ayaty`
- After: language label = `arabizi` (text unchanged)

- Why: the digits 2 and 7 inside Latin words stand for Arabic letters. These reviews need different handling from normal English. Only 9 reviews are Arabizi, so it is rare in this dataset.

4. Problems I faced and how I handled them

#	Problem	How I found it	How I handled it
1	`FileNotFoundError` in Colab	Colab shows uploaded files with spaces in the names, but the notebook used underscores	Used the exact names shown in the Colab Files panel (a glob pattern is a safer alternative)
2	Reviews without text were dropped but are needed for scores	3,265 of 6,120 reviews have no text, yet all have star ratings	Kept two dataframes: `df_all` (6,120, for ratings and place scores) and `df_reviews` (2,855, for NLP)
3	An extra letter after repeat reduction (`جداا`)	Spotted in a before/after example	Split the rule: Arabic reduces 3+ repeats to 1, English caps at 2
4	Arabizi detector gave false positives (35 flagged, only 9 real)	Inspected the flagged reviews. Times such as `9pm`, the word `el` in place names, and English reviews with one Arabizi word were flagged.	Required tokens of 4+ characters, removed ambiguous words (el, fe, fel, mn, aho), and required at least 15% of tokens to be Arabizi
5	`KeyError: 'tokens'`	Cells were run out of order, and re-running `run_pipeline` wipes later columns	Fixed the cell order, added a check cell, and use Restart session and run all

#	Problem	How I found it	How I handled it
6	The saved CSV did not look clean	It had four text versions, and 57% of `text_clean` was identical to the raw text because most reviews had nothing to remove	Added one clear final column, `text_final`, and documented which column to use
7	An old save cell at the end would overwrite the CSV with fewer columns	Found when reviewing the notebook structure	Removed it and kept a single save cell at the end
8	English text inside Arabic reviews	An Arabic review followed by an English translation appeared in the output	The scraper appends a machine translation in the `text` field. I cut it using the `textTranslated` field and a backup rule (blank line, then mostly-Latin text after an Arabic block). 7 reviews fixed.
9	`café` became `caf`	Seen in tokenized output	Extended the tokenizer to keep accented Latin letters
10	Emoji names inside Arabic text (`red_heart`)	Questioned during review	Kept it as a separate column (`text_emoji_text`), so the original emoji stays in `text_clean`. Which version works better is a Milestone 2 experiment.

5. Pipeline v2 (optional experiments, not applied automatically)

The first pipeline cannot finish all the work, so these ideas are implemented as separate columns and will be compared with v1 on the same split in Milestone 2. Only those that improve the metrics will be kept.

#	Idea	Implementation	Result on our data
1	Light Arabic stemming	Rule-based prefix and suffix stripping (`text_stem`)	Works on some words (النكهات to نكه), but it is rough on dialect
2	Dialect-aware normalization	Small Egyptian lexicon (حلوه to حلو, اوي to جدا) in `text_dialect`	Needs a larger lexicon and manual checks
3	Arabizi transliteration	Word lexicon plus character rules (`text_arabizi_ar`)	Crude (for example, to7fa), and only 9 reviews are Arabizi
4	Near-duplicate detection	Cosine similarity of character n-grams (threshold 0.90) in `is_near_duplicate`	9 near-duplicates flagged beyond the exact duplicates
5	Emoji sentiment tags	EMO_POS / EMO_NEG / EMO_NEU in `text_emoji_tag`	To be compared with emoji names in Milestone 2
6	Negation marking	مش حلو becomes مش_حلو in `text_neg`	Needs an experiment to prove it helps
7	Class weights	Balanced class weights from the train split	negative 1.3, neutral 8.23, positive 0.47. Neutral is very rare (112 reviews).
8	Aspect tagging	Rule-based keywords for food, service, price, ambiance, delivery/wait	food 1,286, service 762, ambiance 536, price 194, delivery/wait 168, and 1,017 reviews with no aspect found
9	Review enrichment	Bayesian average score per place using all 6,120 reviews	Places with few reviews no longer get a perfect 5.0 and outrank large places
10	Model-specific tokenization	AraBERT tokenizer	It fragments unsegmented Egyptian text (الاكل becomes الا + ##كل), so Farasa pre-segmentation or a dialect model such as MARBERT should be tested

#	Idea	Implementation	Result on our data
11	Glued-word segmentation	Not implemented	Some reviews contain words glued together (for example رائعةجدا), which the pipeline does not repair yet

6. Limitations and next steps

- **Dataset size:** only 2,855 reviews have text, and 2,415 are usable. This is below the 5,000 mentioned in the project notes, so the minimum should be checked against the full project description.
- **Class imbalance:** neutral reviews are only 112. Handling this is a Milestone 2 experiment.
- **Weak labels:** stars are weak labels (a 4-star review can still contain complaints), so a hand-annotated gold subset is planned.
- **Language detection is based on script:** a review in another Latin-script language (for example Italian) is labeled as English.
- **Arabizi is rare** (9 reviews), and its detection is a heuristic.
- **Next:** compare v1 and v2 columns on the same split, test AraBERT / MARBERT with Farasa segmentation, and build the gold annotation set.

7. Files delivered

- `NLP_pipeline_Yassin_done.ipynb` : the runnable notebook (setup, load and filter, operations 1 to 5, before/after examples, steps 6 to 12, v2 experiments, summary statistics, save).
- `reviews_clean.csv` : 2,855 reviews with the original text, all cleaned versions, language, sentiment, flags and split. Reviewer names, IDs and photos are not included.